

ARM PrimeCell

Vectored Interrupt Controller (PL190)

Integration Manual

ARM PrimeCell Vectored Interrupt Controller (PL190) Integration Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Vectored Interrupt Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	5
4.4	Endianness	5
4.5	Customisation	5
4.5.1	Reduction of Vector Banks	5
4.5.2	Addition of Vector Banks	6
4.6	Synthesis	7
4.6.1	Setup Scripts	7
4.6.2	Command Script	7
4.6.3	Constraint Scripts	8
4.7	Integration with AMBA Micropack components	8
4.8	Interrupt Connection	9
4.8.1	Standalone Connection	9
4.8.2	Daisy Chained Connection	9
4.9	Clocking	10
4.9.1	Bus Clock (HCLK)	10
4.10	Clock Tree synthesis	11
4.11	Resets	11
4.12	Reset Buffering	11
4.13	Integration Testing	11
4.14	Production Testing	11
4.14.1	Scan Insertion and ATPG	12

4.15	Functional and Timing Verification	12
5	APPENDIX A	13
5.1	PrimeCell Environment Variables	13

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A00	13 th June, 2000	First draft
A01	20 th June, 2000	Review comments incorporated
A02	31 st July, 2000	Review comments incorporated
A	8 th August, 2000	First Release

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0181	ARM PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual
3	PL190 DDES 0000	ARM PrimeCell Vectored Interrupt Controller (PL190) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05
7	SCB00-QPRO-0008-A03	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level

VIC	Vectored Interrupt Controller
STA	Static Timing Analysis

2 SCOPE

This document describes how the ARM PrimeCell Vectored Interrupt Controller (VIC PL190) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell VIC is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell VIC (PL190) Technical Reference Manual [Ref.2].

The ARM PrimeCell Vectored Interrupt Controller (PL190) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell VIC block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

The PrimeCell Integration Vector Strategy document [Ref. 7] describes the motivation and use of integration test vectors.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell VIC has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell VIC block.

An example ASIC design flow is described below:

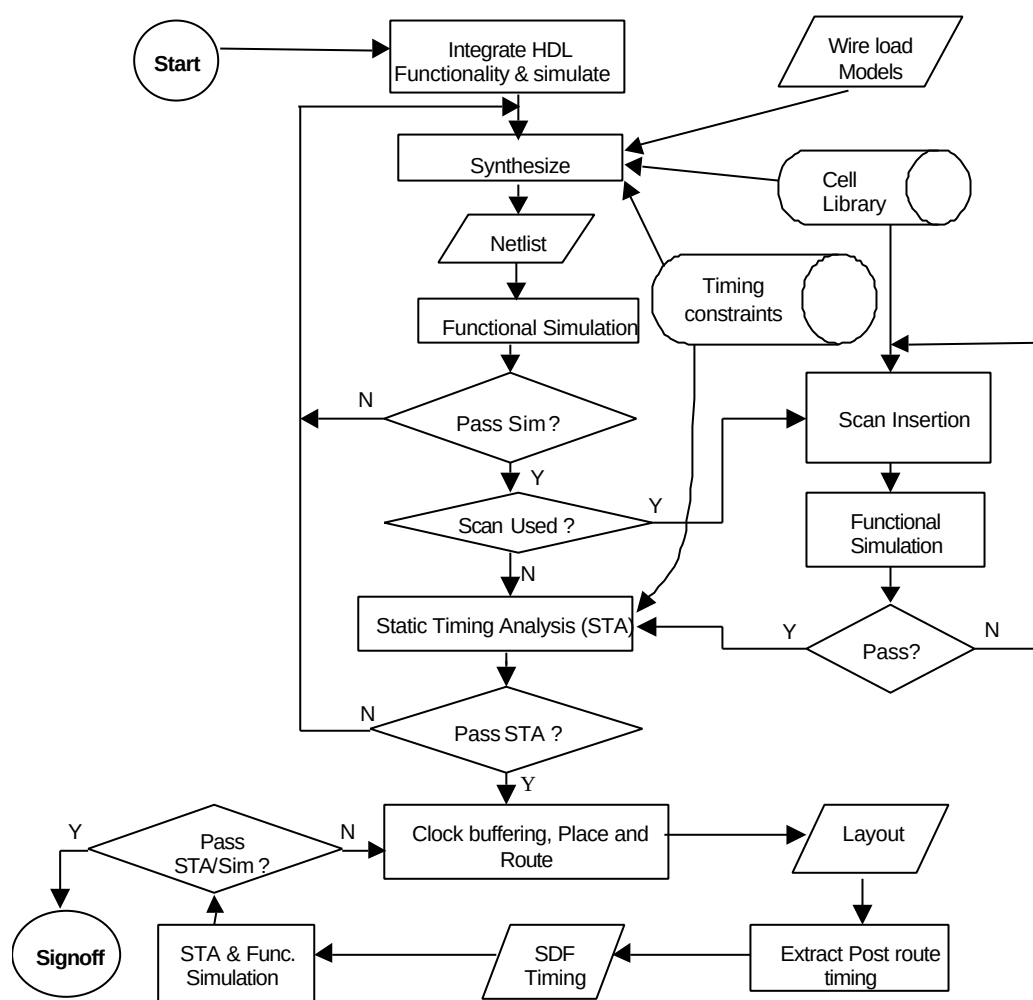


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell VIC block. For further information refer to the ARM PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
HCLK	In	Clock Source	AHB bus clock
HRESETn	In	Reset Controller	AHB bus reset
HADDR[11:2]	In	Master	AHB bus address
HSELVIC	In	Decoder	Vectored Interrupt Controller select
HTRANS	In	Master	AHB bus transfer type. Should be connected to HTRANS[1] of AHB
HPROT	In	Master	AHB bus protection type. Should be connected to HPROT[1] of AHB.
HSIZE[2:0]	In	Master	AHB bus transfer size
HWRITE	In	Master	AHB bus write signal
HWDATA[31:0]	In	Master	AHB write data bus
HRDATA[31:0]	Out	Slave	AHB read data bus
HREADYIN	In	External Slave	AHB bus transfer done indication generated by either the VIC or by an alternate slave.
HREADYOUT	Out	Slave	AHB bus transfer done indication generated by the VIC
HRESP[1:0]	Out	Slave	AHB bus transfer response

Table 4.1-1 AHB interface signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	In	Test Controller	Scan test enable.
SCANINHCLK	In	Test Controller	HCLK domain scan chain input
SCANOUTHCLK	Out	Test Controller	HCLK domain scan chain output
VICINTSOURCE [31:0]	In	Peripheral(s)	Interrupt source input from Peripheral(s)
nVICFIQIN	In	External interrupt controller	Daisy chained FIQ input
nVICIRQIN	In	External interrupt controller	Daisy chained IRQ input
VICVECTADDRIN [31:0]	In	External interrupt controller	Daisy chained interrupt address input
nVICIRQ	Out	ARM / Interrupt Controller	Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain.
nVICFIQ	Out	ARM / Interrupt Controller.	Fast Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain.
VICVECTADDRROUT [31:0]	Out	Interrupt controller	Interrupt address to the next Interrupt Controller in the Daisy chain.

Table 4.2-2: On-Chip Signals

4.3 Bus Interconnection

The VIC block interfaces with the AMBA AHB interface using separate uni-directional read (output) and write (input) data buses HRDATA and HWDATA. For further information refer to the AMBA AHB Specification [Ref.1].

The data buses from the VIC should be interfaced with other data buses from other peripherals using multiplexers at the chip level.

However, it is also possible to interface the VIC block with a 3-state bi-directional data bus. A wrapper needs to be added around the VIC block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the AHB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Endianness

The VIC supports only 32-bit accesses on its AHB data buses. For a 32-bit system, the VIC is bi-endian to the extent that it can support 32-bit accesses from a little-endian master and a big-endian master. In systems with wider data buses, data routing will have to be done external to the VIC.

4.5 Customisation

The ARM PrimeCell VIC can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the ARM PrimeCell Vectored Interrupt Controller (PL190) Design Manual [Ref.3].

4.5.1 Reduction of Vector Banks

The VIC supports 16 Vector Banks. If this is to be reduced, the HDL will have to be modified. An example customisation [using VHDL RTL] where 0 Vector Banks are required is described:

- In the Vic.vhd file, [top level] remove the component VicVectBank and ALL its instantiations.
- In the VicPriority.vhd file, remove VectIrq0Co -> VectIrq15Co, VicVectAddr0 -> VicVectAddr15, PriorWrEnCo, PriorRdEn and PriorityMaskCo from Inputs, Outputs and the body of the HDL code. Remove signals CurrServ, NxtCurrServ, iPriorityMask and ActiveIrq. Remove Processes p_PriMaskComb, p_CurrServSeq, p_CurrServComb and the combinational logic which generates the ActiveIrq vector. The VicPriority module will now only prioritise Non-Vectored Interrupt and the External Interrupt. Hence the NxtVicAddrOut vector is assigned Default Vector address and VICVECTADDRIN when the Non-Vectored and External interrupts are active respectively else the Default Vector address is assigned as follows:

```
NxtVicAddrOut <= VICDefVectAddr when (NonVectIrq = '1')
                else
                VICVECTADDRIN when ((nVICIRQIN = '0') and (NonVectIrq = '0'))
                else
                VICDefVectAddr;
```

Similarly iVICIRQ is to be assigned the logical OR of NonVectIrqCo and inverted nVICIRQIN as follows:

```
iVICIRQ <= ((not nVICIRQIN) and (not NonVectIrqCo)) or (NonVectIrqCo);
```

- In the VicAhbifReg.vhd file, remove VicVectAddr0 -> VicVectAddr15, VicVectCntl0 -> VicVectCntl15 from the Input list and PriorWrEnCo, PriorRdEn, VectAddr0WrEn -> VectAddr15WrEn, VectCntl0WrEn -> VectCntl15WrEn, HWDatInCo and VICIRQStatusCo from the Output list. Remove the signals for address decodes, Read enables and Write enables for the Vector Bank registers and the combinational logic associated with their generation (VicVectAddr0dec -> VicVectAddr15dec, VicVectCntl0dec -> VicVectCntl15dec, VicVectAddr0Rd -> VicVectAddr15Rd, VicVectCntl0Rd -> VicVectCntl15Rd, VectAddr0WrCo -> VectAddr15WrCo, VectCntl0WrCo -> VectCntl15WrCo). The Read data multiplexer should be modified to remove the paths from VicVectAddr0 -> VicVectAddr15 and VicVectCntl0 -> VicVectCntl15.
- In the VicPackage.vhd file, remove Constants for VicVectAddr0 -> VicVectAddr15 and VicVectCntl0 -> VicVectCntl15.
- In the Vic.vhd file, [top level] remove signals VicVectAddr0 -> VicVectAddr15, VicVectCntl0 -> VicVectCntl15, VectAddr0WrCo -> VectAddr15WrCo, VectCntl0WrCo -> VectCntl15WrCo, PriorWrEnCo, PriorRdEn, VICIRQStatusCo, HWDatInCo, PriorityMaskCo and VectIrq0Co -> VectIrq15Co. Reflect the changes in all relevant components and their instantiations.
- No changes are required in the VicRevAnd.vhd file.

Changes to the Verilog RTL can be done on similar lines.

4.5.2 Addition of Vector Banks

The VIC supports 16 Vector Banks. If this is to be increased, the HDL will have to be modified. An example customisation [using VHDL RTL] where 32 Vector Banks are required is described:

- In the Vic.vhd file [top level], add 16 additional instantiations of VicVectBank as u16VicVectBank -> u31VicVectBank.
- In the VicPriority.vhd file, add inputs VectIrq16Co -> VectIrq31Co and VicVectAddr16 -> VicVectAddr31. Convert ActiveIrq, CurrServ, NxtCurrServ and the output PriorityMask to 33 bit vectors. The combinational generation of NxtVicAddrOut is to be modified to add VicVectAddr16 -> VicVectAddr31 [after VicVectAddr15 and before VICDefVectAddr] in sequence of priority qualified by there respective VectIrq signals. Similarly the ActiveIrq signal generation is modified by adding the VectIrq16Co -> VectIrq31Co [after VcetIrq15Co and before NonVectIrqCo] terms. In the combinational generation of iVICIRQ add VectIrq16Co -> VectIrq31Co in the OR tree. In the process p_CurrServComb under the condition (PriorWrEnCo = '1') add the conditions for assigning NxtCurrServ(16) -> NxtCurrServ(31) when the corresponding bit in the CurrServ is active and NxtCurrServ(32) when NonVectIrq is active. In the process p_PriMaskComb add conditions for CurrServ(16) -> CurrServ(31) and assign the 33 bit vector to the PriorityMask vector. The PriorityMask vector should be assigned all Zeros when CurrServ(0) is high and all Ones when all CurrServ bits are low. Care should be taken while making changes to the combinational generation of NxtVicAddrOut, ActiveIrq and processes p_CurrServComb, pPriMaskComb as these are priority encoders.
- In the VicAhbifReg file, add inputs VicVectAddr16 -> VicVectAddr31, VicVectCntl16 -> VicVectCntl31 and outputs VectCntl16WrCo -> VectCntl31WrCo, VectAddr16WrCo -> VectAddr31WrCo. Add signals for address decode, Read enables and Write enables for the added registers (VicVcetAddr16dec -> VicVectAddr31dec, VicVectCntl16dec -> VicVectCntl31dec, VicVectAddr16Rd -> VicVectAddr31Rd, VicVectCntl16Rd -> VicVectCntl31Rd, VectAddr16WrCo -> VectAddr31WrCo, VectCntl16WrCo -> VectCntl31WrCo). In the Read Data multiplexer add paths from VicVectAddr16 -> VicVectAddr31 and VicVectCntl16 -> VicVectCntl31.
- In the VicPackage.vhd file, add constants for addresses of VicVectAddr16 -> VicVectAddr31 and VicVectCntl16 -> VicVectCntl31.
- In the Vic.vhd file [top level], add signals VicVectAddr16 -> VicVectAddr31, VicVectCntl16 -> VicVectCntl31, VectAddr16WrCo -> VectAddr31WrCo, VectCntl16WrCo -> VectCntl31WrCo and VectIrq16Co ->

VectIrq31Co. The PriorityMask vector should be changed to a 32 bit vector. Reflect the changes in all relevant components and their instantiations.

- No changes are required in the VicRevAnd.vhd file.

Changes to the Verilog RTL can be done on similar lines.

4.6 Synthesis

The VIC block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Vic.cmd - synthesizes the VIC to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Vic_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modification to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile command script Vic.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Vic.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script `Vic_scan.cmd` in the `synopsys/scripts` directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in the `library_avanti_cb25_v2.1.scr` file.
- b) Timing constraints for AHB bus signals are set via the `amba_params.scr` and the `ahb_slave.scr` file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The `amba_params.scr` file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume the following bus timing budgets:

- 30 % input setup on HADDR, HWRITE, HSIZE, HWDATA, HSELVIC and HREADYIN
 - 30 % input setup on HTRANS[1] and HPROT[1]
 - 40 % output valid delay on HRDATA, HREADYOUT and HRESP
- c) Timing constraints for non-AMBA VIC-specific signals are set via the `Vic.scr` file. Sample input and output delay constraints are specified for these signals. These constraints may have to be modified based on the actual system configurations – specifically, the number of VICs in a daisy chain will result in large combinational paths from the `nVICFIQIN`, `nVICIRQIN` and `VICVECTADDRIN` to the `nVICFIQ` and `nVICIRQ` outputs. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.
 - d) Timing exceptions are specified via the `Vic_exceptions.scr` file. False paths are specified between the Integration test registers and HRDATA, as these paths are expected to be exercised only during integration testing and not in normal mode of operation. It is not envisaged that Integration tests will be required to be run at full speed (100 MHz).

4.7 Integration with AMBA Micropack components

The VIC block is designed to be integrated into an AMBA system via the AHB (Advanced High-Performance Bus). It is designed to provide a low latency interrupt path from peripherals in the system to the ARM processor. A daisy-chaining capability is also provided to accommodate systems which have more than 32 interrupt sources.

The AHB decoder will need to be altered to take into account the address location of the VIC and any other AHB peripherals in the memory map. The base address of the VIC needs to be defined on a system specific basis and the decode logic altered accordingly. If AMBA TIC methodology is to be used for integration testing then any change of address location for the VIC block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the VIC integration test program.

4.8 Interrupt Connection

The Vectored Interrupt Controller is integrated into the system as an AHB Slave with its nVICIRQ and nVICFIQ outputs connected to the IRQ and FIQ inputs of the ARM processor. Both standalone and daisy-chained configurations are supported.

4.8.1 Standalone Connection

The standalone configuration of the VIC is shown in **Figure 4.8-1**. In this configuration, the nVICIRQIN and nVICFIQIN lines should be tied high and the VICVECTADDRIN[31:0] lines should be tied low.

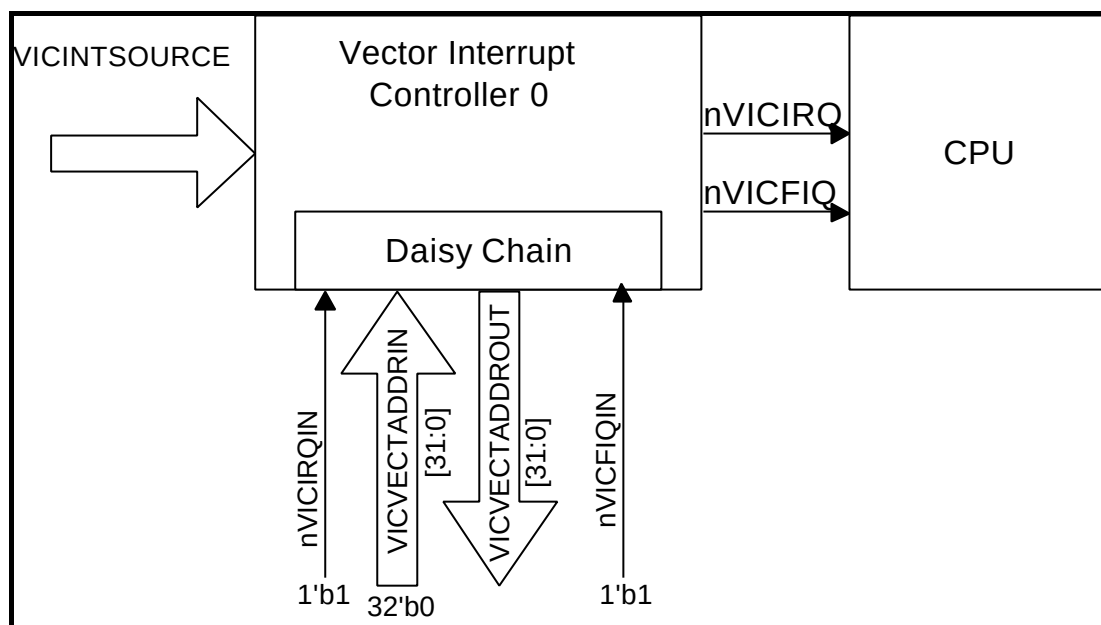


Figure 4.8-1 Standalone Interrupt Controller connectivity

4.8.2 Daisy Chained Connection

The Daisy Chained configuration of the VIC is shown in **Figure 4.8-2**.

In this configuration, the nVICIRQ output of the first VIC should be connected to the nVICIRQIN input of the following interrupt controller. The nVICFIQ output of the first VIC should be connected to the nVICFIQIN input of the following interrupt controller. The VICVECTADDROUT[31:0] output of the first VIC should be connected to the VICVECTADDRIN[31:0] input of the following interrupt controller. The interrupt controller furthest from the central processor should be connected as in the standalone interrupt controller example.

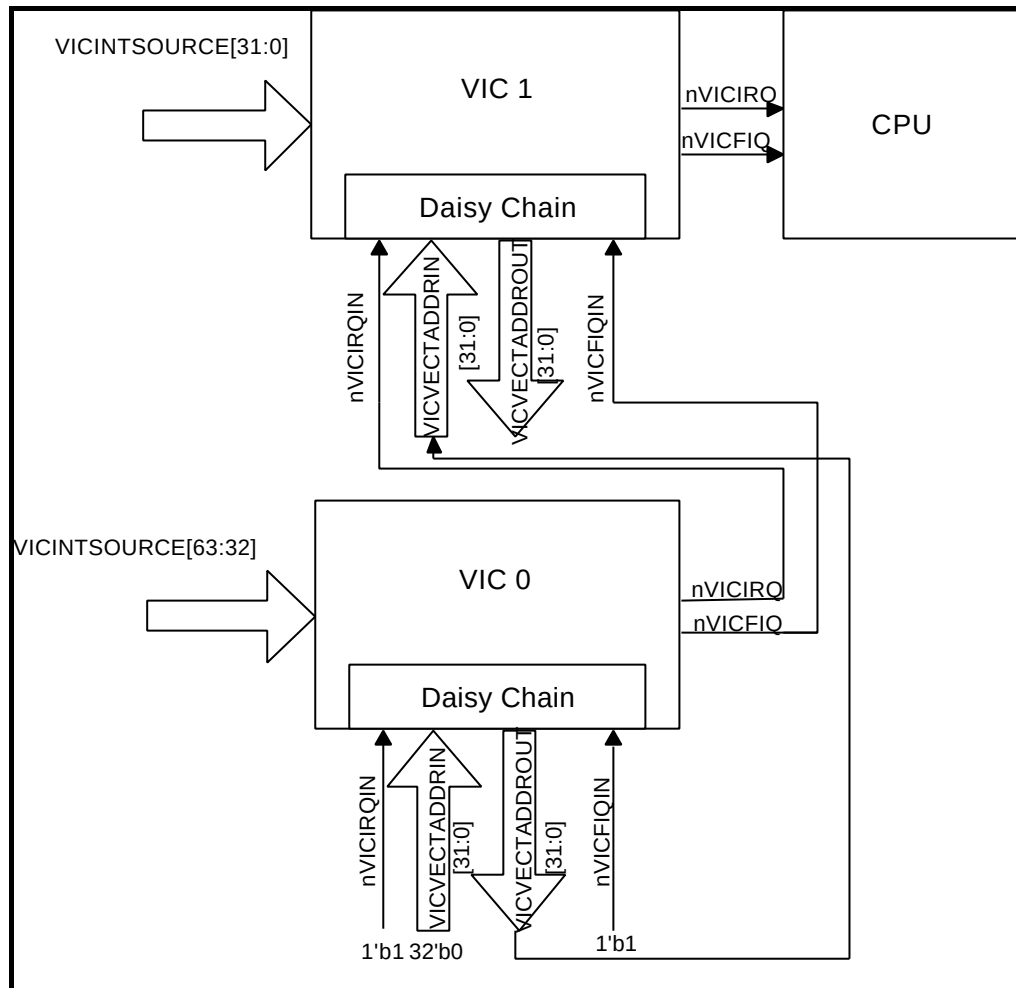


Figure 4.8-2 Daisy Chained Interrupt Controllers connectivity

4.9 Clocking

4.9.1 Bus Clock (HCLK)

The ARM PrimeCell VIC HDL contains a single clock source at the top level – HCLK. This lends itself very well to the ideal single scan chain test strategy.

It should be noted that the VIC design assumes that HCLK is free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical applications. It is imperative that the HCLK is shut off only after the VIC is completely idle. Accesses to the VIC are not guaranteed to complete correctly if HCLK is not free-running. However, a wake-up interrupt facility is supported by keeping the nVICIRQ and nVICFIQ generation paths purely combinational.

4.10 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default VIC synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.11 Resets

The VIC block has a single reset pin. The system bus reset signal HRESETn, resets all internal registers. HRESETn can be asserted asynchronously by the system reset controller but it should be deasserted synchronous to the rising edge of HCLK.

4.12 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.13 Integration Testing

The VIC integration test vector suite allows the integrator to verify that the VIC has been connected into the target system correctly. These vectors test 2 classes of signals:

- a) AMBA signals (VIC signals connected directly to the AHB)
- b) Intra-Chip signals (VIC signals connected to the ARM or to other daisy chained VICs)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The integration vectors are contained in the RegAccessTests.c and IntegrationTests.c files in the integration/bustest directory. As delivered, these vectors are intended to run correctly with the VIC in stand-alone mode, i.e even before the VIC has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the VIC to AHB (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the connectivity of VIC to the ARM and to other Daisy chained VICs will **NEED** modification. The IntegrationTests.c file in the integration/bustest directory has been embedded with NOTES listing the changes that the integrator would have to make in the test code.

4.14 Production Testing

The VIC is designed to be production tested using scan-insertion and ATPG.

4.14.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The VIC design fully supports this scan test and ATPG methodology. The VIC uses only the rising edge of HCLK and hence is synthesised with a single scan chain.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence is again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the VIC block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

4.15 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the Vic world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

eg : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable.

eg: **setenv PERIPH Vic**

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

eg : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

eg : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

eg : **setenv HDL_SOURCE vhdl**
setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

eg : **setenv HDL_NETL vhdl**
setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

eg : **setenv TEST_ENV BUSTALK**
setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

eg : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

eg : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

eg : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

eg: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**